

# 01 · Was ist Echtzeit

Einführung und Paradigmenwechsel · EZS · DHBW Stuttgart

Prof. Dr.-Ing. Daniel Klünder

**Folie**

1 / 51

## Willkommen zu den Echtzeitsystemen!

- Herzlich willkommen im Kurs Echtzeitsysteme (EZS) an der DHBW Stuttgart
- Neun Termine: je 90 Minuten Theorie und 90 Minuten Praxis am Board
- Fokus: moderne Multicore-Architekturen in der Industrie
- Hardware: STM32H755 (Dual-Core Cortex-M7/M4)

## Notizen

Herzlich willkommen zur Vorlesung Echtzeitsysteme. In den nächsten neun Terminen verbinden wir Theorie mit Praxis an Industrie-Hardware. Der STM32H755 dient als Dual-Core-Plattform für Echtzeit und Multicore-Synchronisation.

**Folie**

2 / 51

## Agenda für heute

1. Organisatorisches und Spielregeln
2. Motivation und Definition von Echtzeit
3. Bare-Metal: Polling, Interrupts, Foreground/Background
4. Deep Dive: Cortex-M, NVIC und Tail-Chaining
5. Hardware STM32H755 und Toolchain
6. Laborübung am Board

## Notizen

Zuerst Organisation und Prüfung, dann Motivation und präzise Definition. Anschließend Bare-Metal ohne RTOS, Grenzen von Polling und die Rolle von Interrupts — mit Vorbereitung auf die erste Laborübung.

## Organisatorisches

- Prüfungsleistung: Details folgen (Klausur / Projekt — Angabe in Moodle)
- Materialien: Folien und Code auf der Kurswebsite und in Moodle
- Labor-Hardware: Boards werden gestellt (STM32H755)
- Tools: STM32CubeIDE (C/C++)
- Erwartung: aktive Mitarbeit im Labor — Programmieren lernt man durch Tippen

## Notizen

Alle Folien und Übungsgerüste stehen online. Die Boards bekommen Sie gestellt. Echtzeitverhalten versteht man oft erst, wenn ein Timer-Interrupt nicht rechtzeitig feuert — deshalb Praxispflicht.

## Literatur und Quellen

- Hermann Kopetz: *Real-Time Systems* (Design Principles)
- Giorgio C. Buttazzo: *Hard Real-Time Computing Systems* (Scheduling)
- STMicroelectronics: Reference Manual STM32H745/755 (RM0399)
- FreeRTOS Documentation (online)
- Tipp: Datenblätter und Reference Manuals sind Ihre wichtigste Alltagsquelle

## Notizen

Kopetz und Buttazzo decken Theorie und Scheduling ab. Für die Praxis brauchen Sie ständig das RM0399 — nicht auswendig, aber navigierbar: Timer, Interrupt-Controller, Speicherarchitektur.

## Motivation: Warum Echtzeit?

- Steigende Komplexität in Embedded Systems
- Autonomes Fahren, Medizintechnik, Industrie 4.0
- Ein normaler PC (Windows/Linux) gibt keine harten Garantien
- Timing-Fehler können katastrophale Folgen haben
- Ziel: Systeme, die vorhersagbar reagieren

## Notizen

Wenn der Browser eine halbe Sekunde hängt, ist das ärgerlich. Wenn die Fräsmaschinensteuerung hängt, wird es gefährlich. Es geht nicht um „schnell“, sondern um garantierte, vorhersagbare Reaktionszeiten.

## Folie

6/51

### Der große Mythos der Echtzeit

Was viele denken:

- „Echtzeit = extrem schnell“
- „4 GHz ist echtzeitfähiger als 400 MHz“
- „Echtzeit = flüssiges Video“

Was es wirklich ist:

- Echtzeit = Rechtzeitigkeit
- Ein 400-MHz-Controller mit Garantie kann echtzeitfähig sein
- Ein 4-GHz-PC ohne Garantie ist es oft nicht

## Notizen

Echtzeit hat nichts mit maximalem Durchsatz zu tun. Ein kleiner Mikrocontroller mit fester 10-ms-Reaktion schlägt einen Server, der im Mittel 1 ms braucht, aber gelegentlich 50 ms blockiert.

## Folie

7/51

### Konsequenzen von Timing-Fehlern

- Ausfälle entstehen logisch oder zeitlich
- Beispiel Airbag:
  - Crash erkannt (logisch korrekt)
  - Zündung ausgelöst (logisch korrekt)
  - Zündung 50 ms zu spät (zeitlich falsch)
- Fazit: logisch richtig zur falschen Zeit ist ein Fehler

## Notizen

Beim Airbag reicht korrekte Logik nicht. Zu späte Zündung ist nutzlos oder lebensgefährlich. In Echtzeitsystemen ist ein Ergebnis nach der Deadline ein falsches Ergebnis.

## Lernziele des heutigen Tages

1. Klassische Echtzeit-Definition (Werte- und Zeitbereich) erklären
2. Harte, feste und weiche Echtzeit unterscheiden
3. Metriken Latenz und Jitter verstehen
4. Gefahren von Polling-Loops (Super-Loops) kennen
5. Einfache Hardware-Interrupts einordnen und konfigurieren

## Notizen

Am Ende des Tages: Definition und Klassifikation, Latenz/Jitter, warum `while (1)`-Polling für komplexe Echtzeit ungeeignet ist, und wie Interrupts Determinismus zurückbringen.

## Was ist Echtzeit? (Definition)

- Klassische Definition (häufig nach DIN 44300 zitiert, klassisch / historisch): Echtzeitbetrieb, wenn
  1. Ergebnisse logisch korrekt sind (Wertebereich)
  2. Ergebnisse innerhalb eines vorgegebenen Zeitintervalls vorliegen (Zeitbereich)
- Determinismus: zeitliches Verhalten in jedem Zustand vorhersagbar

## Notizen

Zwei Dimensionen: Wertebereich und Zeitbereich. Die Formulierung geht auf die klassische / historische DIN-44300-Tradition zurück und wird in der Lehre weiterhin so zitiert. Beim Design streben wir Determinismus an — zeitliches Verhalten vorhersagen oder garantieren können.

## Klassifizierung von Echtzeitsystemen

- Nicht jede Fristverletzung ist gleich dramatisch
- Unterscheidung nach Konsequenzen eines Deadline Miss:
  - Harte Echtzeit (Hard Real-Time)
  - Feste Echtzeit (Firm Real-Time)
  - Weiche Echtzeit (Soft Real-Time)
- Grenzen in der Praxis oft fließend und applikationsabhängig

## Notizen

Je nach Kategorie wählen wir Architekturen und Betriebssysteme unterschiedlich. In einem Gerät können alle drei Kategorien auf verschiedene Tasks verteilt sein.

## Folie

11 / 51

### Harte Echtzeit (Hard Real-Time)

- Deadline-Überschreitung hat katastrophale Folgen
- Schäden an Mensch, Umwelt oder extrem hoher Verlust
- Nutzen nach der Deadline: abrupt null oder negativ
- Beispiele: Motorsteuerung, Fly-by-Wire, Herzschrittmacher

## Notizen

Kein Spielraum: Fristverletzung = Systemversagen. Hier müssen wir die Worst-Case Execution Time (WCET) absichern — mathematisch oder durch extreme Tests.

## Folie

12 / 51

### Feste Echtzeit (Firm Real-Time)

- Zwischenstufe zwischen hard und soft
- Keine Katastrophe bei Fristverletzung
- Aber: Ergebnis nach der Deadline völlig wertlos (Nutzen = 0)
- Beispiele: High-Frequency Trading; Ausschuss-Erkennung am Band

## Notizen

Späte Börsenorder: wertlos, System fällt nicht aus. Späte Qualitätsprüfung: Teil ist schon vorbei — Ergebnis verwerfen.

## Weiche Echtzeit (Soft Real-Time)

- Deadline-Überschreitung mindert Qualität, führt aber nicht zum Ausfall
- Ergebnis behält nach der Deadline oft noch Restwert
- Beispiele: Video-Streaming, UI-Verzögerung, Raumtemperaturüberwachung
- Die meisten Standard-IT-Systeme fallen hier hinein

### Notizen

Design oft auf den Average Case und hohen Durchsatz; gelegentliche Ausreißer werden toleriert. Video ruckelt — unschön, aber kein Crash.

## Determinismus und WCET

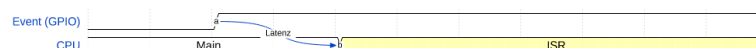
- WCET: maximale, garantierte Ausführungszeit unter Worst-Case-Bedingungen
- BCET: minimale Ausführungszeit
- Harte Echtzeit immer auf Basis der WCET designen
- Caches, Pipeline-Stalls und Interrupts erschweren WCET bei modernen CPUs

### Notizen

Durchschnittswerte reichen für harte Echtzeit nicht. Moderne Features (Caches, Branch Prediction) machen dieselbe Codezeile mal schnell, mal langsam — WCET wird zur Herausforderung.

## Latenz (Reaktionsverzögerung)

- Latenz: Verzögerung zwischen Ereignis und Reaktion
- Interrupt-Latenz: Flanke am Pin bis erste Instruktion der ISR
- Response Time  $R_i$  (Release → Completion): formal erst in VL2; nicht gleich Interrupt-Latenz



### Notizen

Zwischen GPIO-Flanke und ISR: Interrupt erkennen, Pipeline leeren, Register sichern (Exception Entry / Stacking), Sprung in die ISR. Beim Cortex-M oft in der Größenordnung von ca. 12 Taktzyklen Hardware-

Latenz — plus Software. Die Response Time  $R_i$  (Release → Completion) kommt formal erst in VL2 und ist etwas anderes als die Interrupt-Latenz hier.

## Folie

16/51

### Jitter

- Jitter: Varianz der Latenz oder der Periodizität
- Soll alle 10 ms laufen, Ist 9,8 ms / 10,2 ms → Peak-to-Peak-Jitter = 0,4 ms
- Regelungstechnik: hoher Jitter verschlechtert PID-Qualität stark



## Notizen

Schwankendes  $dt$  zerstört Annahmen im Regler. Oft wichtiger als pure Geschwindigkeit: Jitter nahe null, Periodizität stabil.

## Folie

17/51

### Zusammenfassung: Metriken

- Deadline: Zeitpunkt, bis zu dem das Ergebnis vorliegen muss
- WCET: längste Abarbeitungsdauer des Codes
- Latenz: Startverzögerung bis zur Reaktion
- Jitter: Schwankungsbreite der zeitlichen Ausführung
- Goldene Regel: Vorhersagbarkeit (Determinismus), nicht Durchsatz

## Notizen

Fragen zu diesem Theorieblock? Danach: wie man das klassisch programmiert.

## Folie

18/51

### Architekturen: Wie programmieren wir das?

- Wechsel von der Theorie zur Software-Architektur
- Ohne Betriebssystem: Bare-Metal
- Drei klassische Ansätze:
  1. Polling-Loops (Super-Loop)
  2. Interrupt-getriebene Systeme
  3. Foreground/Background-Systeme

## Notizen

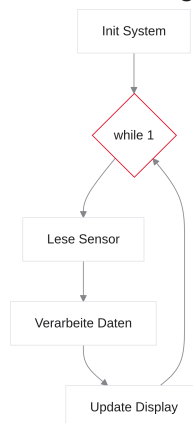
Code läuft direkt auf der Hardware. Die drei Muster sind historisch und didaktisch der Weg zum RTOS.

## Folie

19/51

### Architektur 1: Die Polling-Loop (Super-Loop)

- Simplestes Muster: Endlosschleife `while (1)`
- Tasks strikt sequenziell
- Status wird aktiv abgefragt („Polling“)
- Vorteil: simpel, wenig Overhead, leicht zu debuggen



## Notizen

Klassisches Arduino-Muster: linearer Kontrollfluss. Polling heißt: aktiv nachfragen, ob etwas passiert ist.

## Folie

20/51

### Code-Beispiel: Polling



```

void main(void) {
    System_Init();
    while (1) {
        if (UART_Has_Data()) {    /* Polling */
            Process_Command();
        }
        if (Button_Pressed()) {  /* Polling */
            Toggle_LED();
        }
        Update_Display();
    }
}

```

- Der Code fragt den Status zyklisch ab
- Frage: Was passiert, wenn `Update_Display()` sehr lange dauert?

### Notizen

Während einer langen Display-Funktion ist das System für Taster und UART blind. Latenz summiert sich über alle Tasks in der Schleife.

### Folie

21 / 51

### Die Gefahr: Das Heavy-Load-Problem

- Eine lang laufende Funktion blockiert die gesamte Schleife
- Auf Eingänge kann nicht reagiert werden
- Latenzgarantien sind zerstört
- Typische Sünden: `delay(100)`, blockierende Warteschleifen

```

void Update_Display(void) {
    /* ~50 ms Ausführungszeit */
    Draw_Pixels();
}

```

Worst-Case-Latenz für den Taster:  $> 50 \text{ ms}$

### Notizen

Eine 50-ms-Funktion lässt die Super-Loop stehen. Kurze Tasterimpulse gehen verloren. So darf man in der Industrie nicht programmieren.

## Fazit: Polling

- Vorteile: simpel, keine Race Conditions, feste Reihenfolge
- Nachteile: hoher Stromverbrauch, keine Priorisierung, hohe Latenzen
- Einsatz: nur extrem simple Single-Purpose-Geräte

## Notizen

Für einen dummen Temperatursensor oft ok. Sobald mehrere asynchrone Ereignisse mit unterschiedlicher Dringlichkeit kommen, scheitert das Modell.

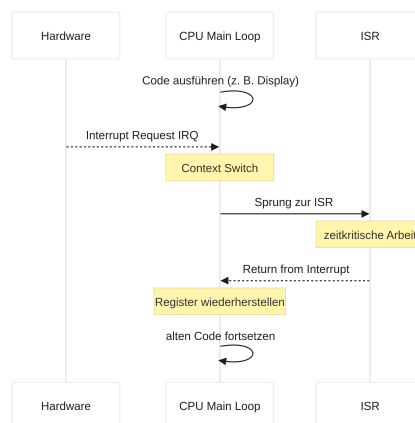
## Architektur 2: Interrupts zur Rettung

- Interrupt: Hardware-Signal, das die CPU unterbricht
- Kontext sichern, Sprung in die ISR
- Nicht mehr fragen — die Hardware meldet sich
- Ereignisse priorisierbar (z. B. Not-Halt vor Display)

## Notizen

Asynchrone Reaktion mit geringer Latenz, unabhängig von der Main-Loop. Das ist der Schlüssel gegen das Heavy-Load-Problem.

## Der Ablauf eines Interrupts



## Notizen

Context Switch sichert Register und Program Counter. Danach ISR, dann Rückkehr genau an die Unterbrechungsstelle.

## Folie

25/51

### Code-Beispiel: Timer-Interrupt (STM32 HAL)

Setup in `main()`:

```
HAL_TIM_Base_Start_IT(&htim2);

while (1) {
    Update_Display(); /* Hintergrund */
}
```

Callback (ISR):

```
void HAL_TIM_PeriodElapsedCallback(
    TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2) {
        Toggle_LED(); /* zeitkritisch */
    }
}
```

## Notizen

`_IT` startet den Timer im Interrupt-Modus. Die LED blinkt deterministisch, auch wenn `Update_Display()` lange braucht.

## Folie

26/51

### Gefahren von Interrupts

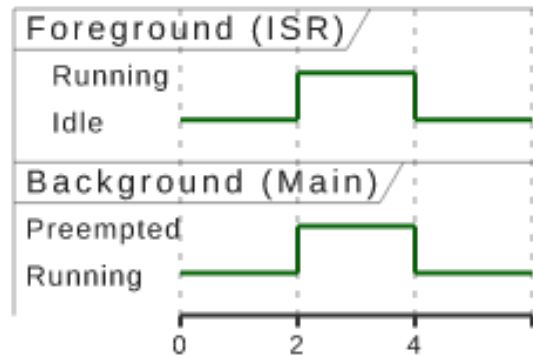
- Race Conditions bei gemeinsamen globalen Variablen
- Lange ISRs blockieren niederpriorige IRQs und die Main-Loop
- Goldene Regel: ISRs extrem kurz halten — kein `printf`, keine Delays
- Jitter: laufende höhere IRQs verzögern niederpriorige ISRs

## Notizen

Asynchronität bringt Korrektheitsfallen. In der ISR: Flag setzen, Wert kopieren, sofort zurück. Lange Arbeit gehört in den Hintergrund oder später in Tasks.

### Architektur 3: Foreground / Background

- Kombination: Main-Loop + Interrupts
- Background: unkritische Polling-Loop (`main`)
- Foreground: zeitkritische ISRs
- Standard für kleine bis mittlere Bare-Metal-Projekte



#### Notizen

ISR präemptiert die Main-Loop. Fertig → Main läuft weiter. Das ist das klassische Bare-Metal-Muster vor dem RTOS.

### Grenzen der klassischen Architekturen

- Warum reicht Foreground/Background oft nicht mehr?
- Spaghetti-Falle: ISRs setzen Flags, Main pollt endlos
- Bei vielen Tasks verliert man den Überblick über den Kontrollfluss
- Wir brauchen Priorisierung und Schlafen von Tasks

#### Notizen

ISR darf nicht lange rechnen → Flag. Main wird Flag-Suppe. Deadlines in der Main-Loop sind dann kaum noch garantierbar.

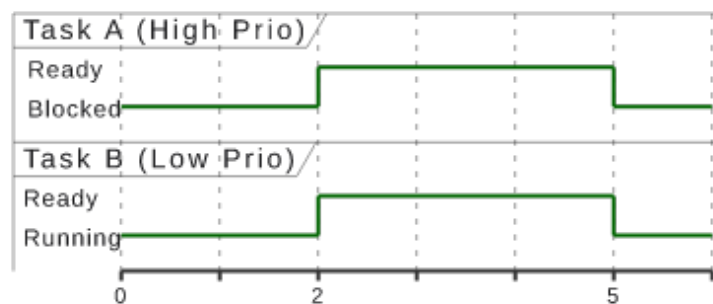
## Der Paradigmenwechsel

- Wenn Bare-Metal nicht skaliert: RTOS (Real-Time Operating System)
- Bare-Metal: Code pollt oder wartet aktiv
- RTOS: Kontrolle über die Zeit an den Scheduler
- Monolith → unabhängige Tasks

### Notizen

Ab hier ändert sich die Denkweise: nicht mehr eine riesige `while (1)` orchestrieren, sondern Tasks und Scheduler. FreeRTOS folgt in späteren Terminen.

## Vorschau: Das Task-Modell



- Task B rechnet; Event weckt Task A
- RTOS entzieht Task B die CPU (Präemption) und gibt sie Task A

### Notizen

Echtes präemptives Multitasking auf einem Core. Details zu Scheduler-Latenzen und Synchronisation: spätere Vorlesungen mit FreeRTOS.

## Deep Dive: Die ARM Cortex-M Architektur

- Um Interrupts zu beherrschen, müssen wir die CPU verstehen
- Der STM32H755 basiert auf ARM Cortex-M (M7 und M4)
- Cortex-M ist für eingebettete Echtzeitsysteme optimiert
- Merkmale:
  - 32-Bit-Architektur (Register, Busse, ALU)
  - Deterministisches Interrupt-Verhalten
  - Keine MMU (virtuelle Adressen), sondern MPU (Memory Protection Unit)

### Notizen

Cortex-M („Microcontroller“) ist auf Determinismus ausgelegt — anders als Cortex-A mit MMU. Physische Adressierung und enge Anbindung des Interrupt-Controllers halten Latenzen vorhersagbar.

## Das Register-Set der CPU

| Register | Rolle                             |
|----------|-----------------------------------|
| R0–R12   | Allgemeine Datenregister          |
| R13 (SP) | Stack Pointer                     |
| R14 (LR) | Link Register (Rücksprungadresse) |
| R15 (PC) | Program Counter                   |
| xPSR     | Statusregister (Flags)            |

- 16 Basis-Register à 32 Bit
- Bei Interrupt: Kontext dieser Register sichern (Hardware-Push)

### Notizen

Bei einem Interrupt sind diese Register voll mit dem Zustand der Main-Loop. Die ISR nutzt dieselben Register — deshalb muss der Kontext gesichert werden. R13 (SP) ist für den Context Switch besonders zentral.

## Was passiert beim Context Switch?

- Interrupt unterbricht die Main-Loop
- CPU muss den Zustand retten, um später fortzusetzen
- Ablauf (hardwaregesteuert):
  1. Pipeline Flush, Abarbeitung stoppt
  2. Automatischer Push: R0–R3, R12, LR, PC, xPSR auf den Stack
  3. PC auf Startadresse der ISR setzen
  4. ISR ausführen
  5. Pop: Register vom Stack wiederherstellen

### Notizen

Der Cortex-M pusht die wichtigsten Register automatisch. Danach erst der Sprung in die C-ISR. Nach dem Rücksprung merkt die Main-Loop die Unterbrechung nicht — sofern die ISR den Stack korrekt hinterlässt.

## Der NVIC: Herr der Interrupts

- NVIC = Nested Vectored Interrupt Controller
- Sitzt im Cortex-M-Kern, eng mit der CPU verzahnt
- Verwaltet viele Interrupt-Quellen (beim STM32 bis zu 240)
- Vectored: jeder Interrupt hat einen Eintrag in der Vector Table
- Hardware springt direkt zur ISR — kein Software-Switch-Case

### Notizen

„Vectored“ spart Latenz: kein allgemeiner Handler mit IF-Kaskade, sondern direkter Sprung über die Vektortabelle.

## Prioritäten im NVIC

- Nested: ein wichtiger Interrupt darf einen unwichtigeren unterbrechen
- Prioritäten als Nummern (z. B. 4 Bit → 0 bis 15)
- Falle: je kleiner die Zahl, desto höher die Priorität
- Priorität 0: höchste Priorität
- Priorität 15: niedrigste Priorität

### Notizen

Lowest number = highest priority. Not-Halt mit Prio 0 muss eine laufende UART-ISR unterbrechen können.

## Preemption Priority vs. Subpriority

- Prioritäts-Bits lassen sich aufteilen (Grouping)
- Preemption Priority: darf ein IRQ einen laufenden IRQ unterbrechen?
- Subpriority: bei gleichzeitigem Auftreten — wer zuerst?
- Höhere Subpriority unterbricht niemals einen bereits laufenden Interrupt

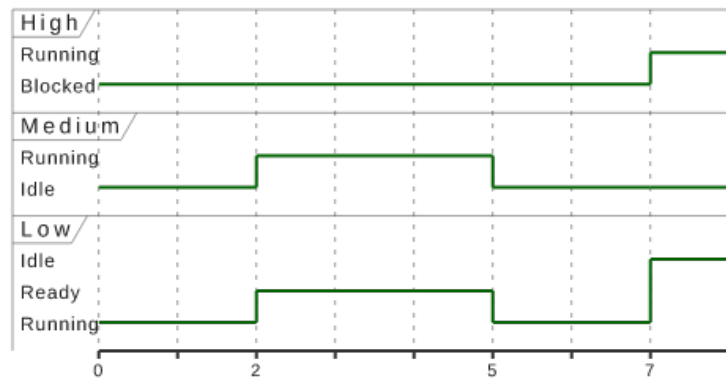
### Notizen

Subpriority entscheidet nur die Warteschlange, nicht die Präemption. In FreeRTOS konfigurieren wir das Grouping typischerweise so, dass wir reine Preemption Priorities nutzen.

## Prioritätsinversion (Teaser)

- Low hält Ressource, High muss warten, Medium präemptiert Low
- Ergebnis: Medium blockiert indirekt High (Priority Inversion)
- Vollständige Behandlung (inkl. Priority Inheritance): VL4





- Teaser: Mars Pathfinder (1997) — Details und Abhilfe in VL4

## Notizen

Low hält die Ressource, High wartet, Medium präemptiert Low — High bleibt blockiert. Das Muster brachte 1997 den Mars Pathfinder in Schwierigkeiten. Vollständige Behandlung (Priority Inheritance Protocol u. a.) folgt in VL4.

## Folie

38 / 51

## Tail-Chaining: ARM's Geheimwaffe

- Zwei Interrupts A und B treten kurz nacheinander auf
- Klassisch: Push → ISR A → Pop → Push → ISR B → Pop
- Tail-Chaining: Cortex-M überspringt sinnloses Pop/Push zwischen ISRs
- Dauer: typisch 6 Taktzyklen statt ca. 24 Zyklen Latenz

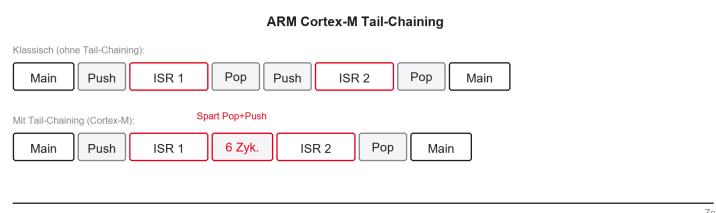


Figure 1: ARM Cortex-M Tail-Chaining: Vergleich mit und ohne Overhead

## Notizen

Wenn nach ISR 1 noch IRQ 2 ansteht, entfällt das teure Pop+Push. Der NVIC kettet die Handler an — Determinismus und Latenzgewinn in Hardware.

## Exkurs Hardware: Der STM32H755

- Dual-Core-Mikrocontroller, Asymmetric Multiprocessing (AMP)
- Cortex-M7 (bis 480 MHz): hohe Leistung, Caches, FPU
- Cortex-M4 (bis 240 MHz): harte Echtzeit, hochdeterministisch
- Beide Cores teilen Chip, Busse und teilweise Speicher

### Notizen

Anders als SMP beim PC: zwei ungleiche Kerne im selben Gehäuse. M7 für Durchsatz, M4 für deterministische Steuerung.

## Warum Dual-Core (AMP) in der Echtzeit?

Cortex-M7 (eher soft):

- Ethernet-Stack (TCP/IP)
- GUI
- aufwändige Filter (DSP)
- Latenz schwankend (Caches)

Cortex-M4 (eher hard):

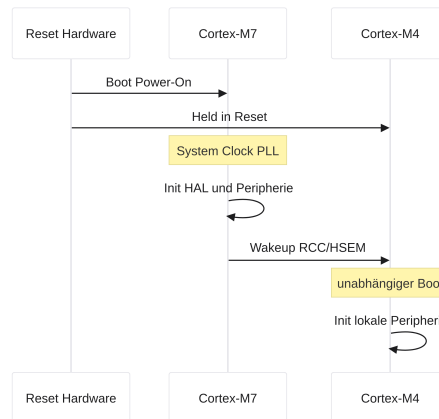
- Motorsteuerung (PWM)
- harte Timing-Loops
- sicherheitskritische Logik
- Latenz hochdeterministisch
- Inter-Core-Kommunikation: Shared Memory im SRAM

### Notizen

Display/Netzwerk und Motorregelung auf einem Kern stören sich gegenseitig. Cache-Misses erzeugen Jitter. AMP trennt die Welten physisch.

## Boot-Sequenz des STM32H755

- Beim Reset starten nicht beide Cores gleichzeitig
- Cortex-M7 bootet als Master
- Cortex-M4 bleibt zunächst im Reset/Sleep
- M7 konfiguriert Systemtakt (PLL), dann weckt M4 (HSEM/RCC)



### Notizen

Beide würden sonst gleichzeitig den Takt konfigurieren. Für Hello World wecken wir den M4 und nutzen ihn als Arbeitskern.

## Einführung in die Tools: STM32CubeIDE

- STM32CubeIDE: Eclipse-basierte C/C++-IDE für STM32
- Device Configuration Tool (ehemals CubeMX)
- Grafische Pin-Zuweisung und Clock-Tree
- Codegenerator für die HAL

### Notizen

Statt hundert Seiten Register-Doku: Pin wählen, Funktion zuweisen, Takt konfigurieren — der Generator liefert die Basis-Initialisierung.

## Das HAL-Konzept

- Nicht direkt in Register tippen (Standardfall)
- HAL: lesbar und oft portabel innerhalb der STM32-Familie
- Nachteil: leichter Aufruf-Overhead
- Extreme Echtzeit: LL oder direkte Register

```
/* HAL (Standard) */
HAL_TIM_Base_Start_IT(&htim2);

/* LL (schneller) */
LL_TIM_EnableIT_UPDATE(TIM2);
LL_TIM_EnableCounter(TIM2);

/* Register */
TIM2->DIER |= (1u << 0);
TIM2->CR1  |= (1u << 0);
```

### Notizen

Für den Großteil der Vorlesung reicht HAL. Wenn jede Nanosekunde zählt, wechseln wir gezielt auf LL oder Registerzugriff.

## Projektstruktur und Code-Generierung

- Pro Core eigener Ordner (CM4 / CM7)
- Je eigene `main.c` und `stm32h7xx_it.c`
- Eigener Code nur zwischen `USER CODE BEGIN` und `USER CODE END`
- Code außerhalb dieser Blöcke wird beim erneuten Generieren gelöscht

### Notizen

Die wichtigste Anfängerregel: nur in USER-CODE-Blöcken schreiben — sonst ist die Arbeit mit einem Speichern in der GUI weg.

## Laboraufgabe 1: Hello World (LED Blink)

- Ziel: Toolchain prüfen, Cortex-M4 nutzen
- Schritte:
  1. Projekt anlegen (Board STM32H755ZI-Q)
  2. LED (z. B. PE1) als GPIO\_Output für CM4
  3. Code generieren
  4. In `main.c` (CM4) toggeln

```
/* USER CODE BEGIN 3 */
while (1) {
    HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_1);
    HAL_Delay(500); /* blockiert 500 ms */
}
/* USER CODE END 3 */
```

### Notizen

Wenn die LED blinkt, laufen IDE, Flash und Debugger. Das Delay ist bewusst blockierend — als Kontrast zu Aufgabe 3.

## Laboraufgabe 2: Das Polling-Desaster

- Ziel: Grenzen der Super-Loop spüren
- `Heavy_Load()` vor dem Toggle aufrufen (simuliert Filter/Display)

```
while (1) {
    Heavy_Load(800000); /* CPU-Last */
    HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_1);
    HAL_Delay(500);
}
```

- Beobachtung: Warum ist der 500-ms-Takt zerstört?

### Notizen

Die unbekannte Laufzeit von `Heavy_Load()` addiert sich zum Delay. Latenz und Periodizität werden unvorhersagbar — genau das Heavy-Load-Problem aus der Theorie.

### Laboraufgabe 3: Rettung durch den Timer

- Ziel: Determinismus per Interrupt wiederherstellen
- In der `.ioc`: TIM2 aktivieren, Clock Source Internal
- NVIC: „TIM2 global interrupt“ aktivieren
- Timer im Interrupt-Modus starten
- Periodizität berechnen, z. B. 500 ms (2 Hz):

$$f_{\text{UPD}} = \frac{f_{\text{CLK}}}{(PSC + 1)(ARR + 1)}$$

#### Notizen

Ohne NVIC-Haken feuert die Hardware den IRQ nicht zur CPU. PSC und ARR aus dem tatsächlichen Timer-Takt ableiten.

### Laboraufgabe 3: Der Callback-Code

- Toggle und Delay aus der `while(1)` entfernen — dort nur noch `Heavy_Load()`
- Callback ans Ende von `main.c`:

```
/* USER CODE BEGIN 4 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2) {
        HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_1);
    }
}
/* USER CODE END 4 */
```

- Ergebnis: LED blinkt präzise, unabhängig von der CPU-Last

#### Notizen

Foreground/Background: ISR toggelt die LED, Main rechnet Heavy Load. Präzises Blinken trotz Last.

### Laboraufgabe 4 (Bonus): Messung am Oszi

- Ziel: Jitter Main-Loop vs. Timer-ISR messen
- Freien GPIO (z. B. PA0) als Output nutzen
- Ablauf: Pin High  $\rightarrow$  Heavy\_Load ( )  $\rightarrow$  Pin Low; Pulsweite am Oszi
- Vergleich: Pin-Toggle in der ISR (Jitter  $\approx$  0)

### Notizen

In der Industrie glauben wir Timing nicht, wir messen es. WCET der Berechnung vs. fast jitterfreie ISR-Flanke.

### Zusammenfassung des Tages

- Echtzeit = Rechtzeitigkeit, nicht pure Rechenleistung
- Super-Loops skalieren schlecht (Latenz-Akkumulation)
- Interrupts ermöglichen Hardware-Präemption (Foreground/Background)
- NVIC: Prioritäten hardwarenah, inkl. Tail-Chaining
- Workflow: Bare-Metal-Code auf Cortex-M4 generieren und erweitern

### Notizen

Determinismus und Deadlines stehen im Zentrum. Interrupts retten Latenz — mit neuen Gefahren (Race Conditions, lange ISRs). Hardware-Features wie Tail-Chaining senken Overhead.

### Ausblick auf Termin 2

- Bare-Metal mit vielen IRQs skaliert nicht ewig (Spaghetti-Flags)
- Nächste Woche: Scheduling-Theorie
- Praxis: eigener einfacher Round-Robin-Scheduler auf dem STM32
- Vorbereitung: heutiges Projekt mitbringen

### Notizen

Danach der Weg zu FreeRTOS. Bitte Projekte speichern — wir bauen darauf auf. Fragen? Dann weiter im Labor.